

Blockchain-Based Secure Platform for Identity, Access Control, and Digital Asset Management

Complete Solution Document

Organization: Bharat Electronics Limited (BEL) | Theme: Blockchain & Cybersecurity

This document is the complete technical solution for PS 26125. It covers system architecture, every module's implementation detail, workflows, smart-contract design, the full mapping of every stated, unstated, and future-facing problem to a concrete engineered solution, deployment strategy, and the phased implementation roadmap. It is written to be directly usable as the technical section of the SIH submission and as an implementation blueprint for the actual build.

1. Executive Summary

Our solution is a five-module blockchain platform — Decentralized Identity, Smart-Contract-Governed RBAC, NFT-based Digital Asset Management, Immutable Audit Trail with AI-assisted monitoring, and Multi-Signature Governance with Dispute Resolution — deployed first on a public test network for the hackathon prototype, with an explicit production migration path to a permissioned consortium chain suited to BEL's defense-sector compliance needs.

Every module is designed against a specific, named threat model — not just the problems BEL stated in the PS, but the deeper architectural flaws such systems inherently carry (admin key compromise, key-loss lockout, irreversible contract bugs, oracle trust gaps, legal-enforceability gaps) and forward-looking risks such as quantum-computer-breakable cryptography. Each subsequent section explains what a module is, exactly how it is implemented, and how it works end-to-end.

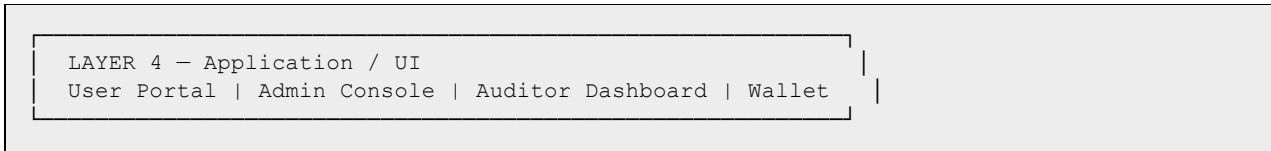
1.1 Five Core Modules

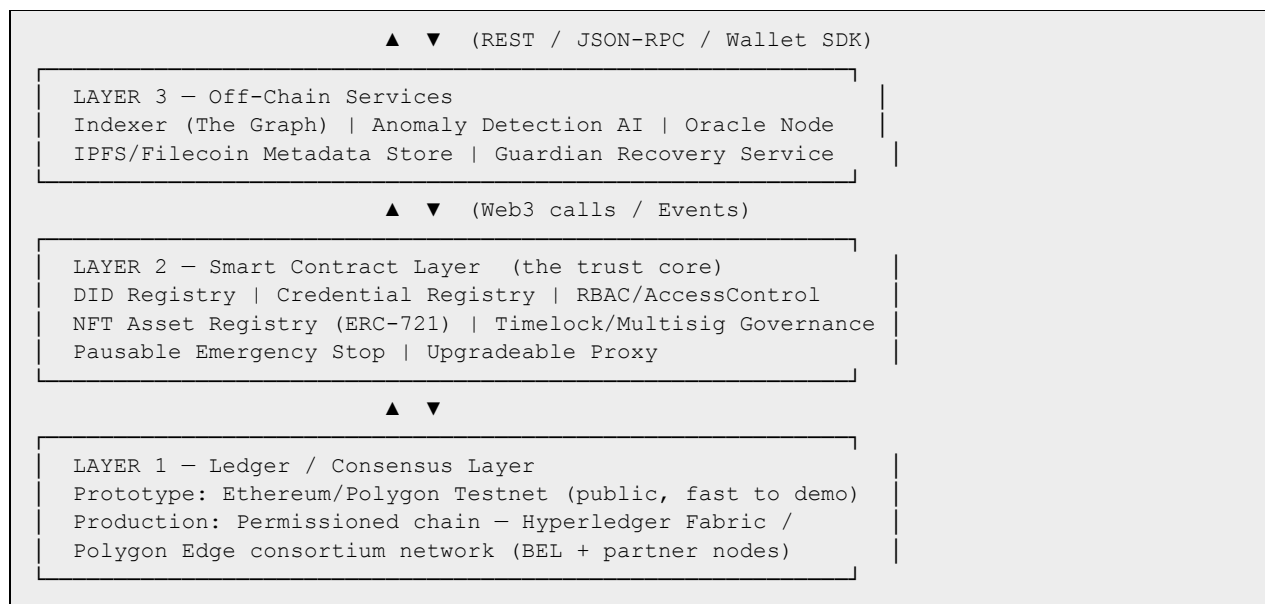
Module	Solves
M1 — Decentralized Identity (DID) & Credentials	Centralized IAM risk, identity theft, key loss, Sybil attacks, privacy leakage
M2 — Smart-Contract RBAC Engine	Unauthorized access, admin key compromise, instant revocation, contract bugs, gas cost
M3 — NFT Asset Registry	Disconnected ownership records, authenticity, metadata rot, legal enforceability
M4 — Immutable Audit Trail + Anomaly Detection	Transparency, tamper-proof history, real-time misuse detection
M5 — Multi-Sig Governance & Dispute Resolution	'Who watches the admin', irreversible fraud, single point of trust

2. System Architecture Overview

The platform is organized into four horizontal layers. Each layer only talks to the layer directly above/below it, which keeps the system auditable and lets any single layer (e.g., the chain itself) be swapped later (public testnet → permissioned chain) without rewriting the modules above it.

2.1 Layered Architecture





2.2 Design Principle: Crypto-Agility

Every cryptographic operation (signature verification, hashing) is called through an interface contract, never hard-coded inline. This is what makes Section 10 (quantum resistance) possible without a full system rewrite later — the signature scheme can be swapped behind the same interface.

3. Technology Stack

Layer	Prototype (Hackathon)	Production Target
Ledger	Ethereum Sepolia / Polygon Mumbai testnet	Hyperledger Fabric or Polygon Edge (permissioned)
Smart contracts	Solidity 0.8.x + OpenZeppelin (AccessControl, Pausable, UUPS Proxy, ERC-721)	Same, audited + formally verified
Identity standard	W3C DID Core + Verifiable Credentials (did:ethr / did:key)	did:web / org-specific DID method
Off-chain storage	IPFS (via Pinata/Web3.Storage)	IPFS + Filecoin with SLA-backed pinning
Indexing	The Graph subgraph	Self-hosted Graph node (data sovereignty)
Frontend	React + Ethers.js/Wagmi + WalletConnect	Same, integrated with BEL SSO for onboarding
Anomaly detection	Python service (rule-based + lightweight ML) consuming indexed events	Full ML pipeline with SOC integration
Zero-Knowledge layer	Semaphore / snarkjs (proof-of-role without identity reveal)	Production-grade zk-SNARK circuits (audited)

4. Module 1 — Decentralized Identity (DID) & Verifiable Credentials

This module replaces the centralized user database entirely. Every user, device, and even organizational department is represented as a DID — a cryptographically self-owned identifier — with real-world trust established through Verifiable Credentials issued by BEL, not by the mere existence of the DID itself.

4.1 What It Is

DID: A W3C-standard identifier of the form `did:ethr:0xABC...` resolving to a DID Document containing the user's public key(s) and service endpoints. It lives on-chain (DID Registry contract) and is controlled only by the matching private key.

Verifiable Credential (VC): A signed JSON object issued by an authorized BEL credential-issuer DID, stating a fact about a subject DID — e.g., "DID X holds the role Manager, valid until 2027-01-01, issued by DID(BEL-HR-Admin)". VCs are held by the user off-chain (wallet) and only their revocation status is checked on-chain.

4.2 How It Is Implemented

1. Onboarding: HR/IT verifies the employee's real-world identity (existing internal process) and generates a fresh key pair for them inside a hardware-backed wallet (or software wallet for prototype).
2. DID creation: the public key is registered on-chain via `DIDRegistry.createDID(pubKey, metadataURI)`, emitting a `DIDCreated` event.
3. Credential issuance: an authorized Issuer role signs a VC binding the new DID to the employee's role/department and pushes only its hash + revocation-registry entry on-chain (the VC payload itself stays off-chain / in the user's wallet for privacy).
4. Authentication: to log in or authorize an action, the user signs a challenge nonce with their private key; the dApp / contract verifies the signature against the DID Document's registered public key — no password ever exists.

4.3 Solving the Sub-Problems

Sub-problem	Implementation detail
Private key loss (Section 2.1.1 of gap analysis)	Social recovery module: user pre-registers 3–5 guardian DIDs (e.g., HR admin + IT admin + one peer). Losing the key triggers a <code>GuardianRecovery</code> contract requiring M-of-N (e.g., 3-of-5) guardian signatures within a time-locked window to rotate the DID's registered public key to a new one — the DID itself, and everything linked to it, survives.
Sybil attacks	DIDs are functionally worthless without a VC from an authorized BEL issuer; role-gated and asset-gated actions check for a valid, non-revoked VC, not merely DID existence, so fake DIDs cannot acquire privileges.
DID ≠ real identity	VC issuance is bound to BEL's existing verified-onboarding process (HR record check) — the credential, not the raw DID, is what the system trusts, closing the gap explicitly.
Privacy / permanent correlation	Zero-knowledge proof of credential possession (via Semaphore) lets a user prove "I hold a valid Auditor VC" to a contract without revealing which specific DID or wallet address they are, breaking the public correlation chain for sensitive actions.
Quantum readiness (forward-looking)	Signature verification is routed through an <code>ISignatureVerifier</code> interface; DID Documents store an algorithm identifier alongside the public key so a future DID can register a Dilithium key instead of ECDSA with zero change to the registry contract logic. See Section 10.

5. Module 2 — Smart-Contract-Governed RBAC Engine

Access control is not a database flag checked by application code (which can be bypassed) — it is enforced inside the smart contract itself, so every privileged function call reverts automatically unless the caller's DID holds the required, non-expired role.

5.1 Role Hierarchy

Role	Typical Permissions
Super Admin (multisig)	Deploy/upgrade contracts, add/remove Admins, trigger emergency pause
Admin	Mint NFTs, assign roles (within policy limits), issue credentials
Manager	Approve asset allocation, co-sign dual-attestation mints
Auditor	Read-only access to full audit trail, flag disputes, no write privileges
User	View own identity/assets, initiate transfer requests (subject to approval)

5.2 How It Is Implemented

- Built on OpenZeppelin AccessControl, extended with an expiry timestamp per (DID, role) pair — not just a boolean grant.
- hasRole() is overridden to check both "role granted" AND "block.timestamp < expiryOf[account][role]" — an expired role silently stops passing checks without requiring any extra transaction, solving the instant-revocation-latency problem.
- High-privilege actions (minting, role grants above User level) require a 2-of-3 multisig from Admin-tier DIDs via a Gnosis-Safe-style module, not a single Admin key — directly mitigating admin-key compromise.
- A Pausable modifier on all state-changing functions lets any 2 Super Admins freeze the entire contract within one transaction if a compromise is detected, buying time to investigate without funds/assets moving.
- The RBAC contract sits behind a UUPS upgradeable proxy — logic bugs can be patched by deploying a new implementation and having governance approve the upgrade, without losing any on-chain role/asset state.

5.3 Sample Contract Pattern (illustrative Solidity)

```
contract TimeBoundAccessControl is AccessControlUpgradeable, PausableUpgradeable {
    mapping(bytes32 => mapping(address => uint256)) public roleExpiry;

    function grantTimedRole(bytes32 role, address account, uint256 validUntil)
        external onlyRole(getRoleAdmin(role)) whenNotPaused {
        _grantRole(role, account);
        roleExpiry[role][account] = validUntil;
        emit TimedRoleGranted(role, account, validUntil);
    }

    function hasRole(bytes32 role, address account)
        public view override returns (bool) {
        return super.hasRole(role, account)
            && block.timestamp < roleExpiry[role][account];
    }
}
```

6. Module 3 — NFT-Based Digital Asset Registry

Every asset (digital file, license, credential, or a tokenized representation of a physical item) is minted as an ERC-721 NFT, directly linked to the owning DID, with metadata content-addressed on IPFS so the token cannot silently point to altered content.

6.1 How It Is Implemented

5. Metadata for the asset (description, documents, images, provenance data) is uploaded to IPFS; the resulting content hash (CID) is what gets stored on-chain, not a mutable URL.
6. Minting requires dual attestation: an Admin proposes the mint, a Manager (or second Admin) must co-sign within a defined window before `AssetRegistry.mint()` actually executes — directly mitigating the "anyone can mint an NFT for an asset they don't possess" flaw.
7. The minted NFT's owner field is set to the recipient's DID-linked wallet address; ownership transfer functions check the recipient DID holds a valid, non-revoked identity credential before allowing transfer, so assets cannot be sent to an unverified/anonymous address.
8. A `legalReference` field in the metadata schema stores a hash of an off-chain signed attestation document, giving a path for the token to be recognized as evidence in a legal dispute once bridged to a recognized framework — explicitly future-proofing against the legal-enforceability gap identified in the analysis.

6.2 Solving the Sub-Problems

Sub-problem	Solution
Metadata rot / mutable pointers	IPFS content-addressing: the CID is a hash of the content itself, so any tampering changes the hash and is instantly detectable by re-fetching and comparing.
Fake/unauthorized minting of physical assets	Mandatory dual attestation (two independent roles must co-sign before mint finalizes).
Legal ownership gap	<code>legalReference</code> hash field + off-chain signed attestation, designed to plug into a future legal-tech bridge without a schema change.
Duplicate assets	ERC-721 standard guarantees per-token uniqueness at the contract level; a <code>tokenId</code> can never be minted twice.

7. Module 4 — Immutable Audit Trail & Anomaly Detection

Every state-changing function across every module emits a structured event. Nothing is logged by application code alone — if it isn't an on-chain event, it isn't trusted as "the record."

7.1 How It Is Implemented

- Every contract emits typed events: `DIDCreated`, `CredentialIssued`, `CredentialRevoked`, `RoleGranted`, `RoleExpired`, `AssetMinted`, `AssetTransferred`, `EmergencyPaused`, `DisputeRaised`, etc., each carrying actor DID, target, timestamp, and relevant metadata hash.
- A Graph Protocol subgraph indexes these events into a queryable API, powering the Auditor Dashboard (read-only role, per Section 5.1) without needing to trust any centralized backend — anyone can re-index from the raw chain data and get the same result.
- An off-chain Anomaly Detection service subscribes to the indexed event stream and applies rule-based + lightweight ML checks: unusual mint velocity, role escalation outside business hours, repeated failed-auth attempts, mass transfer patterns — flags surface as risk-scored alerts in the Admin/Auditor console, feeding Section 8's dispute process.

8. Module 5 — Multi-Signature Governance & Dispute Resolution

This module answers "who watches the Admin" directly: no single Admin key can unilaterally perform a high-value action, and even a legitimately-signed-but-fraudulent transaction has a window in which it can be challenged and reversed.

8.1 How It Is Implemented

- Governance contract modeled on Gnosis Safe: Super Admin actions (contract upgrades, adding/removing Admins, emergency pause/unpause) require M-of-N multisig approval, with N and M configured by BEL's own policy (e.g., 3-of-5 department heads).
- High-value asset transfers pass through a TimelockController: the transfer is queued, becomes executable only after a cooling-off period (e.g., 24–48 hours), and during that window any Auditor-role DID can `raiseDispute()`, which freezes the specific queued transaction pending Super Admin multisig review — reversible before finalization, immutable after.
- All governance actions are themselves logged through Module 4's event system, so even governance itself is fully auditable.

9. End-to-End Workflows

9.1 Workflow — New Employee Onboarding (Identity Creation)

9. HR verifies the employee's real-world identity through existing internal process.
10. Employee's wallet generates a key pair locally (private key never leaves the device).
11. Public key + metadata submitted to `DIDRegistry.createDID()` → `DIDCreated` event fires.
12. Employee registers 3–5 guardians for future key-recovery.
13. Authorized HR-Issuer DID signs a Verifiable Credential stating role and department; VC hash + revocation-registry entry pushed on-chain, full VC handed to employee's wallet.
14. Employee can now authenticate anywhere in the system by signing a challenge — no password created, none exists to steal.

9.2 Workflow — Minting and Allocating a Digital Asset

15. Admin uploads asset metadata + documents to IPFS, receives a CID.
16. Admin calls `proposeMint(CID, recipientDID)` — creates a pending mint request, does not yet create the token.
17. A second authorized role (Manager) reviews and calls `coSignMint(requestId)` — dual attestation satisfied.
18. `AssetRegistry.mint()` executes automatically once both signatures are present, emitting `AssetMinted`; the NFT appears in the recipient's wallet, permanently linked to their DID.
19. Event is indexed by the subgraph and immediately visible on the Auditor Dashboard.

9.3 Workflow — Role Revocation (Employee Termination)

20. HR/Admin calls `revokeRole(role, employeeDID)` — role removed instantly, no waiting on expiry.
21. In parallel, `roleExpiry` is set to `block.timestamp`, so even a queued/pending transaction from that DID fails the `hasRole()` check on its next execution.
22. `CredentialRegistry` marks the associated VC as revoked; any zero-knowledge proof attempt referencing that credential will fail verification from this point forward.
23. `RoleExpired` / `CredentialRevoked` events fire, visible instantly on the audit trail.

9.4 Workflow — Detecting and Freezing a Compromised Admin Key

24. Anomaly Detection service flags unusual behavior (e.g., rapid sequential role grants at 3 AM) from an Admin DID.
25. Alert surfaces to Auditor + remaining Super Admins.
26. Any 2 Super Admins call `emergencyPause()` — all state-changing functions revert platform-wide within a single transaction.
27. Investigation proceeds off-chain; once resolved, the compromised DID's key is rotated via the guardian-recovery flow (Section 4.3) and the contract is unpaused by a fresh multisig vote.

10. Complete Problem → Solution Mapping

This table consolidates every problem identified across the earlier gap-analysis document — stated, unstated, and future-facing — against the specific module and mechanism that resolves it.

10.1 Stated Problems (from the Official PS)

Problem	Module	Mechanism
Centralized IAM single point of failure	M1	DIDs anchored on-chain; no master identity server
Identity theft / cyberattacks	M1	Cryptographic challenge-response auth; no passwords exist
Unauthorized access	M2	Contract-enforced RBAC with time-bound roles
Disconnected asset ownership records	M3	Single shared NFT registry linked to DIDs
Ownership/authenticity hard to verify	M3 + M4	On-chain provenance + immutable event log

10.2 Unstated Problems We Identified

Problem	Module	Mechanism
Private key loss = permanent lockout	M1	M-of-N guardian social recovery
Sybil identity creation	M1	VC-gated privileges, not raw DID existence
DID ≠ verified real identity	M1	VC issuance bound to HR-verified onboarding
Public correlation / privacy leakage	M1	Zero-knowledge proof of credential possession
Admin key compromise = full takeover	M2 + M5	Multisig for high-privilege actions + emergency pause
No instant access revocation	M2	Time-bound roles + instant roleExpiry override
Irreversible smart contract bugs	M2	UUPS upgradeable proxy pattern
Gas cost / scalability at org scale	M2	Layer-2 deployment (Polygon rollup)
Oracle / off-chain trust gap	M3	Dual-attestation minting; multi-attestor oracle design
NFT ownership ≠ legal ownership	M3	legalReference hash field for future legal-tech bridge
Off-chain metadata rot	M3	IPFS content-addressed storage
Fake physical-asset minting	M3	Dual attestation requirement
"Who watches the Admin"	M5	Multisig governance + full audit logging of governance itself
No dispute resolution for fraud	M5	Timelock cooling-off window + Auditor veto
Regulatory / data-localization blindness	Deployment	Permissioned consortium chain for production (Section 11)

10.3 Future-Facing Problems

Problem	Module	Mechanism
Quantum computers breaking ECDSA	M1 + M2	Crypto-agile signature interface (Section 11)
Sophisticated real-time misuse patterns	M4	AI anomaly detection service
Cross-organization identity portability	M1	Interoperable DID method (did:web) for future chain bridges
Compliance-proof without data exposure	M1	Zero-knowledge compliance proofs (future roadmap)

11. Quantum-Resistance & Crypto-Agility Design

Rather than claiming to be "quantum-proof" today (no current production blockchain honestly is), our architecture is built to be crypto-agile — capable of migrating cryptographic primitives without a system rewrite, which is the realistic and credible engineering answer to this threat.

11.1 The Threat

- ECDSA (used for DID keys and transaction signing on Ethereum/Polygon) is breakable by Shor's algorithm on a cryptographically-relevant quantum computer.
- Because all on-chain data is public and permanent, an adversary can record public keys and signatures today and break them retroactively once quantum hardware matures — the exposure window has already started, regardless of when the hardware arrives ("harvest now, decrypt later").

11.2 The Implementation Answer

- 28. Every signature check across all contracts routes through a single ISignatureVerifier interface rather than inline ecrecover() calls.
- 29. DID Documents store a keyType field (e.g., "ES256K" today) alongside the public key.
- 30. When NIST-standardized post-quantum schemes (CRYSTALS-Dilithium for signatures, CRYSTALS-Kyber for key exchange — both finalized by NIST in 2024) have mature, audited implementations for EVM-compatible chains, a new key type can be registered per-DID and the verifier swapped via the governed upgrade path (Section 5.2) — no identity, asset, or role data is lost or needs re-issuing.
- 31. New DIDs can optionally register a hybrid key (classical + post-quantum signature required together) ahead of full migration, for early risk reduction on the highest-privilege Super Admin accounts.

12. Deployment Architecture

12.1 Prototype (Hackathon Demo)

- Deployed on a public EVM testnet (Polygon Amoy / Ethereum Sepolia) for fast, verifiable, judge-accessible demonstration.
- Frontend hosted separately (Vercel/Netlify); IPFS metadata via a pinning service (Pinata/Web3.Storage) for reliability during the demo.

12.2 Production (Recommended for BEL)

- Migrate the same contract logic to a permissioned consortium chain — Hyperledger Fabric (channel-based privacy, enterprise-grade permissioning) or a private Polygon Edge network — with validating nodes run by BEL and any partner PSUs, addressing data-localization and classified-handling requirements the public-testnet prototype cannot satisfy.
- Layer-2 rollup or app-chain scaling adopted once transaction volume grows beyond pilot scale, keeping per-transaction cost predictable at organizational scale.
- Self-hosted Graph node instead of the public hosted service, keeping indexed audit data inside BEL's own infrastructure boundary.

13. Security Architecture Summary

Threat	Defense Layer
Stolen/lost private key	Guardian-based social recovery (M1)
Compromised Admin account	Multisig for privileged actions + emergency pause (M2/M5)
Malicious/buggy contract logic	Upgradeable proxy + external audit before mainnet

Threat	Defense Layer
Fraudulent but validly-signed transaction	Timelock cooling-off + Auditor veto (M5)
Fake asset minting	Dual attestation requirement (M3)
Metadata tampering	IPFS content-addressing (M3)
Sybil identity spam	VC-gated, issuer-authorized credentialing (M1)
Future quantum decryption	Crypto-agile signature interface (Section 11)

14. Implementation Roadmap

Phase	Scope
Phase 1 — Hackathon MVP	DID registry, VC issuance, time-bound RBAC, ERC-721 asset minting with dual attestation, basic audit dashboard, testnet deployment
Phase 2 — Pilot Hardening	Multisig governance, timelock dispute resolution, guardian recovery, IPFS metadata pipeline, external contract audit
Phase 3 — Production Migration	Permissioned chain deployment, L2 scaling, self-hosted indexing, AI anomaly detection service, ZK privacy layer
Phase 4 — Forward-Looking	Post-quantum signature migration path activated, legal-tech bridge integration, cross-organization DID interoperability

15. Conclusion

This solution treats PS 26125 as more than a DID + NFT + RBAC checklist. Every module is built against a named, specific failure mode — including ones the problem statement never mentions — and the architecture is deliberately staged so today's public-testnet prototype has a credible, engineered path to a real BEL production deployment: permissioned consensus, legal enforceability, quantum-safe cryptography, and governed, auditable trust at every layer, rather than merely relocating a single point of failure from a database to an admin key.